

After lines 2–7					After line 8				
<i>i</i>	<i>left-rank</i>	<i>right-rank</i>	<i>index</i>	substring	<i>i</i>	<i>left-rank</i>	<i>right-rank</i>	<i>index</i>	substring
1	114	97	1	ra	1	97	116	2	at
2	97	116	2	at	2	97	116	4	at
3	116	97	3	ta	3	97	116	6	at
4	97	116	4	at	4	114	97	1	ra
5	116	97	5	ta	5	116	0	7	t
6	97	116	6	at	6	116	97	3	ta
7	116	0	7	t	7	116	97	5	ta

Figure 32.12 The *substr-rank* array for indices $i = 1, 2, \dots, 7$ after the **for** loop of lines 2–7 and after the sorting step in line 8 for input string $T = \text{ratatat}$.

$\text{ord}(\text{b}) = 2, \dots, \text{ord}(\text{z}) = 26$ would work. Once the substrings being considered contain multiple characters, their ranks will be positive integers less than or equal to n , coming from their relative order after being sorted. An empty substring always has rank 0, since it is lexicographically less than any nonempty substring.

The COMPUTE-SUFFIX-ARRAY procedure uses objects internally to keep track of the relative ordering of the substrings according to their ranks. When considering substrings of a given length, the procedure creates and sorts an array *substr-rank*[1 : n] of n objects, each with the following attributes:

- *left-rank* contains the rank of the left part of the substring.
- *right-rank* contains the rank of the right part of the substring.
- *index* contains the index into the text T of where the substring starts.

Before delving into the details of how the procedure works, let's look at how it operates on the input text *ratatat*, with $n = 7$. Assuming that the *ord* function returns the ASCII code for a character, Figure 32.12 shows the *substr-rank* array after the **for** loop of lines 2–7 and then after the sorting step in line 8. The *left-rank* and *right-rank* values after lines 2–7 are the ranks of length-1 substrings in positions i and $i + 1$, for $i = 1, 2, \dots, n$. These initial ranks are the ASCII values of the characters. At this point, the *left-rank* and *right-rank* values give the ranks of the left and right part of each substring of length 2. Because the substring starting at index 7 consists of only one character, its right part is empty and so its *right-rank* is 0. After the sorting step in line 8, the *substr-rank* array gives the relative lexicographic order of all the substrings of length 2, with starting points of these substrings in the *index* attribute. For example, the lexicographically smallest length-2 substring is *at*, which starts at position *substr-rank*[1].*index*, which equals 2. This substring also occurs at positions *substr-rank*[2].*index* = 4 and *substr-rank*[3].*index* = 6.

The procedure then enters the **while** loop of lines 10–19. The loop variable l gives an upper bound on the length of substrings that have been sorted thus far.

After line 11		After lines 12–17					After line 18				
<i>i</i>	<i>rank</i>	<i>i</i>	<i>left-rank</i>	<i>right-rank</i>	<i>index</i>	<i>substring</i>	<i>i</i>	<i>left-rank</i>	<i>right-rank</i>	<i>index</i>	<i>substring</i>
1	2	1	2	4	1	rata	1	1	0	6	at
2	1	2	1	1	2	atat	2	1	1	2	atat
3	4	3	4	4	3	tata	3	1	1	4	atat
4	1	4	1	1	4	atat	4	2	4	1	rata
5	4	5	4	3	5	tat	5	3	0	7	t
6	1	6	1	0	6	at	6	4	3	5	tat
7	3	7	3	0	7	t	7	4	4	3	tata

Figure 32.13 The *rank* array after line 11 and the *substr-rank* array after lines 12–17 and after line 18 in the first iteration of the **while** loop of lines 10–19, where $l = 2$.

Entering the **while** loop, therefore, the substrings of length at most $l = 2$ are sorted. The call of MAKE-RANKS in line 11 gives each of these substrings its rank in the sorted order, from 1 up to the number of unique length-2 substrings, based on the values it finds in the *substr-rank* array. With $l = 2$, MAKE-RANKS sets *rank*[*i*] to be the rank of the length-2 substring $T[i : i + 1]$. Figure 32.13 shows these new ranks, which are not necessarily unique. For example, since the length-2 substring *at* occurs at positions 2, 4, and 6, MAKE-RANKS finds that *substr-rank*[1], *substr-rank*[2], and *substr-rank*[3] have equal values in *left-rank* and in *right-rank*. Since *substr-rank*[1].*index* = 2, *substr-rank*[2].*index* = 4, and *substr-rank*[3].*index* = 6, and since *at* is the smallest substring in lexicographic order, MAKE-RANKS sets *rank*[2] = *rank*[4] = *rank*[6] = 1.

This iteration of the **while** loop will sort the substrings of length at most 4 based on the ranks from sorting the substrings of length at most 2. The **for** loop of lines 12–17 reconstitutes the *substr-rank* array, with *substr-rank*[*i*].*left-rank* based on *rank*[*i*] (the rank of the length-2 substring $T[i : i + 1]$) and *substr-rank*[*i*].*right-rank* based on *rank*[*i* + 2] (the rank of the length-2 substring $T[i + 2 : i + 3]$, which is 0 if this substring starts beyond the end of the length-*n* text). Together, these two ranks give the relative rank of the length-4 substring $T[i : i + 3]$. Figure 32.13 shows the effect of lines 12–17. The figure also shows the result of sorting the *substr-rank* array in line 18, based on the *left-rank* attribute, and using the *right-rank* attribute to break ties. Now *substr-rank* gives the lexicographically sorted order of all substrings with length at most 4.

The next iteration of the **while** loop, with $l = 4$, sorts the substrings of length at most 8 based on the ranks from sorting the substrings of length at most⁴ 4. Figure 32.14 shows the ranks of the length-4 substrings and the *substr-rank* array

⁴ Why keep saying “length at most”? Because for a given value of l , a substring of length l starting at position i is $T[i : i + l - 1]$. If $i + l - 1 > n$, then the substring cuts off at the end of the text.

After line 11		After lines 12–17					After line 18				
<i>i</i>	rank	<i>i</i>	left-rank	right-rank	index	substring	<i>i</i>	left-rank	right-rank	index	substring
1	3	1	3	5	1	ratatat	1	1	0	6	at
2	2	2	2	1	2	atatat	2	2	0	4	atat
3	6	3	6	4	3	tatat	3	2	1	2	atatat
4	2	4	2	0	4	atat	4	3	5	1	ratatat
5	5	5	5	0	5	tat	5	4	0	7	t
6	1	6	1	0	6	at	6	5	0	5	tat
7	4	7	4	0	7	t	7	6	4	3	tatat

Figure 32.14 The *rank* array after line 11 and the *substr-rank* array after lines 12–17 and after line 18 in the second—and final—iteration of the **while** loop of lines 10–19, where $l = 4$.

before and after sorting. This iteration is the final one, since with the length n of the text equaling 7, the procedure has sorted all substrings.

In general, as the loop variable l increases, more and more of the right parts of the substrings are empty. Therefore, more of the *right-rank* values are 0. Because i is at most n within the loop of lines 12–17, the left part of each substring is always nonempty, and so all *left-rank* values are always positive.

This example illuminates why the COMPUTE-SUFFIX-ARRAY procedure works. The initial ranks established in lines 2–7 are simply the ord values of the characters in the text, and so when line 8 sorts the *substr-rank* array, its ordering corresponds to the lexicographic ordering of the length-2 substrings. Each iteration of the **while** loop of lines 10–19 takes sorted substrings of length l and produces sorted substrings of length $2l$. Once l reaches or exceeds n , all substrings have been sorted.

Within an iteration of the **while** loop, the MAKE-RANKS procedure “re-ranks” the substrings that were sorted, either by line 8 before the first iteration or by line 18 in the previous iteration. MAKE-RANKS takes a *substr-rank* array, which has been sorted, and fills in an array $rank[1 : n]$ so that $rank[i]$ is the rank of the i th substring represented in the *substr-rank* array. Each rank is a positive integer, starting from 1, and going up to the number of unique substrings of length $2l$. Substrings with equal values of *left-rank* and *right-rank* receive the same rank. Otherwise, a substring that is lexicographically smaller than another appears earlier in the *substr-rank* array, and it receives a smaller rank. Once the substrings of length $2l$ are re-ranked, line 18 sorts them by rank, preparing for the next iteration of the **while** loop.

Once l reaches or exceeds n and all substrings are sorted, the values in the *index* attributes give the starting positions of the sorted substrings. These indices are precisely the values that constitute the suffix array.

Let’s analyze the running time of COMPUTE-SUFFIX-ARRAY. Lines 1–7 take $\Theta(n)$ time. Line 8 takes $O(n \lg n)$ time, using either merge sort (see Section 2.3.1) or heapsort (see Chapter 6). Because the value of l doubles in each iteration of

the **while** loop of lines 10–19, this loop makes $\lceil \lg n \rceil - 1$ iterations. Within each iteration, the call of **MAKE-RANKS** takes $\Theta(n)$ time, as does the **for** loop of lines 12–17. Line 18, like line 8, takes $O(n \lg n)$ time, using either merge sort or heap-sort. Finally, the **for** loop of lines 20–21 takes $\Theta(n)$ time. The total time works out to $O(n \lg^2 n)$.

A simple observation allows us to reduce the running time to $\Theta(n \lg n)$. The values of *left-rank* and *right-rank* being sorted in line 18 are always integers in the range 0 to n . Therefore, radix sort can sort the *substr-rank* array in $\Theta(n)$ time by first running counting sort (see Chapter 8) based on *right-rank* and then running counting sort based on *left-rank*. Now each iteration of the **while** loop of lines 10–19 takes only $\Theta(n)$ time, giving a total time of $\Theta(n \lg n)$.

Exercise 32.5-2 asks you to make a simple modification to **COMPUTE-SUFFIX-ARRAY** that allows the **while** loop of lines 10–19 to iterate fewer than $\lceil \lg n \rceil - 1$ times for certain inputs.

Computing the *LCP* array

Recall that $LCP[i]$ is defined as the length of the longest common prefix of the $(i - 1)$ st and i th lexicographically smallest suffixes $T[SA[i - 1] :]$ and $T[SA[i] :]$. Because $T[SA[1] :]$ is the lexicographically smallest suffix, we define $LCP[1]$ to be 0.

In order to compute the *LCP* array, we need an array *rank* that is the inverse of the *SA* array, just like the final *rank* array in **COMPUTE-SUFFIX-ARRAY**: if $SA[i] = j$, then $rank[j] = i$. That is, we have $rank[SA[i]] = i$ for $i = 1, 2, \dots, n$. For a suffix $T[i :]$, the value of $rank[i]$ gives the position of this suffix in the lexicographically sorted order. Figure 32.11 includes the *rank* array for the *ratatat* example. For example, the suffix *tat* is $T[5 :]$. To find this suffix's position in the sorted order, look up $rank[5] = 6$.

To compute the *LCP* array, we will need to determine where in the lexicographically sorted order a suffix appears, but with its first character removed. The *rank* array helps. Consider the i th smallest suffix, which is $T[SA[i] :]$. Dropping its first character gives the suffix $T[SA[i] + 1 :]$, that is, the suffix starting at position $SA[i] + 1$ in the text. The location of this suffix in the sorted order is given by $rank[SA[i] + 1]$. For example, for the suffix *atat*, let's see where to find *tat* (*atat* with its first character removed) in the lexicographically sorted order. The suffix *atat* appears in position 2 of the suffix array, and $SA[2] = 4$. Thus, $rank[SA[2] + 1] = rank[5] = 6$, and sure enough the suffix *tat* appears in location 6 in the sorted order.

The procedure **COMPUTE-LCP** on the next page produces the *LCP* array. The following lemma helps show that the procedure is correct.

```

COMPUTE-LCP( $T, SA, n$ )
1  allocate arrays  $rank[1 : n]$  and  $LCP[1 : n]$ 
2  for  $i = 1$  to  $n$ 
3       $rank[SA[i]] = i$                 // by definition
4   $LCP[1] = 0$                         // also by definition
5   $l = 0$                             // initialize length of LCP
6  for  $i = 1$  to  $n$ 
7      if  $rank[i] > 1$ 
8           $j = SA[rank[i] - 1]$  //  $T[j : ]$  precedes  $T[i : ]$  lexicographically
9           $m = \max\{i, j\}$ 
10         while  $m + l \leq n$  and  $T[i + l] == T[j + l]$ 
11              $l = l + 1$         // next character is in common prefix
12          $LCP[rank[i]] = l$     // length of LCP of  $T[j : ]$  and  $T[i : ]$ 
13         if  $l > 0$ 
14              $l = l - 1$         // peel off first character of common prefix
15 return  $LCP$ 

```

Lemma 32.8

Consider suffixes $T[i - 1 :]$ and $T[i :]$, which appear at positions $rank[i - 1]$ and $rank[i]$, respectively, in the lexicographically sorted order of suffixes. If $LCP[rank[i - 1]] = l > 1$, then the suffix $T[i :]$, which is $T[i - 1 :]$ with its first character removed, has $LCP[rank[i]] \geq l - 1$.

Proof The suffix $T[i - 1 :]$ appears at position $rank[i - 1]$ in the lexicographically sorted order. The suffix immediately preceding it in the sorted order appears at position $rank[i - 1] - 1$ and is $T[SA[rank[i - 1] - 1] :]$. By assumption and the definition of the LCP array, these two suffixes, $T[SA[rank[i - 1] - 1] :]$ and $T[i - 1 :]$, have a longest common prefix of length $l > 1$. Removing the first character from each of these suffixes gives the suffixes $T[SA[rank[i - 1] - 1] + 1 :]$ and $T[i :]$, respectively. These suffixes have a longest common prefix of length $l - 1$. If $T[SA[rank[i - 1] - 1] + 1 :]$ immediately precedes $T[i :]$ in the lexicographically sorted order (that is, if $rank[SA[rank[i - 1] - 1] + 1] = rank[i] - 1$), then the lemma is proven.

So now assume that $T[SA[rank[i - 1] - 1] + 1 :]$ does not immediately precede $T[i :]$ in the sorted order. Since $T[SA[rank[i - 1] - 1] :]$ immediately precedes $T[i - 1 :]$ and they have the same first $l > 1$ characters, $T[SA[rank[i - 1] - 1] + 1 :]$ must appear in the sorted order somewhere before $T[i :]$, with one or more other suffixes intervening. Each of these suffixes must start with the same $l - 1$ characters as $T[SA[rank[i - 1] - 1] + 1 :]$ and $T[i :]$, for otherwise it would appear either before

$T[SA[rank[i - 1] - 1] + 1 :]$ or after $T[i :]$. Therefore, whichever suffix appears in position $rank[i] - 1$, immediately before $T[i :]$, has at least its first $l - 1$ characters in common with $T[i :]$. Thus, $LCP[rank[i]] \geq l - 1$. ■

The COMPUTE-LCP procedure works as follows. After allocating the *rank* and *LCP* arrays in line 1, lines 2–3 fill in the *rank* array and line 4 pegs *LCP*[1] to 0, per the definition of the *LCP* array.

The **for** loop of lines 6–14 fills in the rest of the *LCP* array going by decreasing-length suffixes. That is, it fills the position of the *LCP* array in the order $rank[1]$, $rank[2]$, $rank[3]$, $\dots$, $rank[n]$, with the assignment occurring in line 12. Upon considering a suffix $T[i :]$, line 8 determines the suffix $T[j :]$ that immediately precedes $T[i :]$ in the lexicographically sorted order. At this point, the longest common prefix of $T[j :]$ and $T[i :]$ has length at least l . This property certainly holds upon the first iteration of the **for** loop, when $l = 0$. Assuming that line 12 sets $LCP[rank[i]]$ correctly, line 14 (which decrements l if it is positive) and Lemma 32.8 maintain this property for the next iteration. The longest common prefix of $T[j :]$ and $T[i :]$ might be even longer than the value of l at the start of the iteration, however. Lines 9–11 increment l for each additional character the prefixes have in common so that it achieves the length of the longest common prefix. The index m is set in line 9 and used in the test in line 10 to make sure that the test $T[i + l] == T[j + l]$ for extending the longest common prefix does not run off the end of the text T . When the **while** loop of lines 10–11 terminates, l is the length of the longest common prefix of $T[j :]$ and $T[i :]$.

As a simple aggregate analysis shows, the COMPUTE-LCP procedure runs in $\Theta(n)$ time. Each of the two **for** loops iterates n times, and so it remains only to bound the total number of iterations by the **while** loop of lines 10–11. Each iteration increases l by 1, and the test $m + l \leq n$ ensures that l is always less than n . Because l has an initial value of 0 and decreases at most $n - 1$ times in line 14, line 11 increments l fewer than $2n$ times. Thus, COMPUTE-LCP takes $\Theta(n)$ time.

Exercises

32.5-1

Show the *substr-rank* and *rank* arrays before each iteration of the **while** loop of lines 10–19 and after the last iteration of the **while** loop, the suffix array *SA* returned, and the sorted suffixes when COMPUTE-SUFFIX-ARRAY is run on the text *hippityhoppity*. Use the position of each letter in the alphabet as its ord value, so that $\text{ord}(\text{b}) = 2$. Then show the *LCP* array after each iteration of the **for** loop of lines 6–14 of COMPUTE-LCP given the text *hippityhoppity* and its suffix array.

32.5-2

For some inputs, the COMPUTE-SUFFIX-ARRAY procedure can produce the correct result with fewer than $\lceil \lg n \rceil - 1$ iterations of the **while** loop of lines 10–19. Modify COMPUTE-SUFFIX-ARRAY (and, if necessary, MAKE-RANKS) so that the procedure can stop before making all $\lceil \lg n \rceil - 1$ iterations in some cases. Describe an input that allows the procedure to make $O(1)$ iterations. Describe an input that forces the procedure to make the maximum number of iterations.

32.5-3

Given two texts, T_1 of length n_1 and T_2 of length n_2 , show how to use the suffix array and longest common prefix array to find all of the *longest common substrings*, that is, the longest substrings that appear in both T_1 and T_2 . Your algorithm should run in $O(n \lg n + kl)$ time, where $n = n_1 + n_2$ and there are k such longest substrings, each with length l .

32.5-4

Professor Markram proposes the following method to find the longest palindromes in a string $T[1:n]$ by using its suffix array and LCP array. (Recall from Problem 14-2 that a palindrome is a nonempty string that reads the same forward and backward.)

Let $@$ be a character that does not appear in T . Construct the text T' as the concatenation of T , $@$, and the reverse of T . Denote the length of T' by $n' = 2n + 1$. Create the suffix array SA and LCP array LCP for T' . Since the indices for a palindrome and its reverse appear in consecutive positions in the suffix array, find the entries with the maximum LCP value $LCP[i]$ such that $SA[i - 1] = n' - SA[i] - LCP[i] + 2$. (This constraint prevents a substring—and its reverse—from being construed as a palindrome unless it really is one.) For each such index i , one of the longest palindromes is $T'[SA[i] : SA[i] + LCP[i] - 1]$.

For example, if the text T is *unreferenced*, with $n = 12$, then the text T' is *unreferenced@decnerefernu*, with $n' = 25$ and the following suffix array and LCP array:

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
$T'[i]$	u	n	r	e	f	e	r	e	n	c	e	d	@	d	e	c	n	e	r	e	f	e	r	n	u
$SA[i]$	13	10	16	12	14	15	11	4	20	8	18	6	22	5	21	9	17	2	24	3	19	7	23	25	1
$LCP[i]$	0	0	1	0	1	0	1	1	4	1	1	3	2	0	3	0	1	1	1	0	5	2	1	0	1

The maximum LCP value is achieved at $LCP[21] = 5$, and $SA[20] = 3 = n' - SA[21] - LCP[21] + 2$. The suffixes of T' starting at indices $SA[20]$ and $SA[21]$ are *referenced@decnerefernu* and *refernu*, both of which start with the length-5 palindrome *refer*.

Alas, this method is not foolproof. Give an input string T that causes this method to give results that are shorter than the longest palindrome contained within T , and explain why your input causes the method to fail.

Problems

32-1 String matching based on repetition factors

Let y^i denote the concatenation of string y with itself i times. For example, $(ab)^3 = ababab$. We say that a string $x \in \Sigma^*$ has **repetition factor** r if $x = y^r$ for some string $y \in \Sigma^*$ and some $r > 0$. Let $\rho(x)$ denote the largest r such that x has repetition factor r .

- a. Give an efficient algorithm that takes as input a pattern $P[1:m]$ and computes the value $\rho(P[1:i])$ for $i = 1, 2, \dots, m$. What is the running time of your algorithm?
- b. For any pattern $P[1:m]$, let $\rho^*(P)$ be defined as $\max \{\rho(P[1:i]) : 1 \leq i \leq m\}$. Prove that if the pattern P is chosen randomly from the set of all binary strings of length m , then the expected value of $\rho^*(P)$ is $O(1)$.
- c. Argue that the procedure REPETITION-MATCHER correctly finds all occurrences of pattern $P[1:m]$ in text $T[1:n]$ in $O(\rho^*(P)n + m)$ time. (This algorithm is due to Galil and Seiferas. By extending these ideas greatly, they obtained a linear-time string-matching algorithm that uses only $O(1)$ storage beyond what is required for P and T .)

REPETITION-MATCHER(T, P, n, m)

```

1   $k = 1 + \rho^*(P)$ 
2   $q = 0$ 
3   $s = 0$ 
4  while  $s \leq n - m$ 
5      if  $T[s + q + 1] == P[q + 1]$ 
6           $q = q + 1$ 
7          if  $q == m$ 
8              print "Pattern occurs with shift"  $s$ 
9          if  $q == m$  or  $T[s + q + 1] \neq P[q + 1]$ 
10              $s = s + \max \{1, \lceil q/k \rceil\}$ 
11              $q = 0$ 
```


32-2 A linear-time suffix-array algorithm

In this problem, you will develop and analyze a linear-time divide-and-conquer algorithm to compute the suffix array of a text $T[1:n]$. As in Section 32.5, assume that each character in the text is represented by an underlying encoding, which is a positive integer.

The idea behind the linear-time algorithm is to compute the suffix array for the suffixes starting at $2/3$ of the positions in the text, recursing as needed, use the resulting information to sort the suffixes starting at the remaining $1/3$ of the positions, and then merge the sorted information in linear time to produce the full suffix array.

For $i = 1, 2, \dots, n$, if $i \bmod 3$ equals 1 or 2, then i is a *sample position*, and the suffixes starting at such positions are *sample suffixes*. Positions 3, 6, 9, $\dots$ are *nonsample positions*, and the suffixes starting at nonsample positions are *nonsample suffixes*.

The algorithm sorts the sample suffixes, sorts the nonsample suffixes (aided by the result of sorting the sample suffixes), and merges the sorted sample and nonsample suffixes. Using the example text $T = \text{bippityboppityboo}$, here is the algorithm in detail, listing substeps of each of the above steps:

1. The sample suffixes comprise about $2/3$ of the suffixes. Sort them by the following substeps, which work with a heavily modified version of T and may require recursion. In part (a) of this problem on page 999, you will show that the orders of the suffixes of T and the suffixes of the modified version of T are the same.

- A. Construct two texts P_1 and P_2 made up of “metacharacters” that are actually substrings of three consecutive characters from T . We delimit each such metacharacter with parentheses. Construct

$$P_1 = (T[1:3]) (T[4:6]) (T[7:9]) \cdots (T[n':n' + 2]) ,$$

where n' is the largest integer congruent to 1, modulo 3, that is less than or equal to n and T is extended beyond position n with the special character $\emptyset$, with encoding 0. With the example text $T = \text{bippityboppityboo}$, we get that

$$P_1 = (\text{bip}) (\text{pit}) (\text{ybo}) (\text{ppi}) (\text{tyb}) (\text{oo}\emptyset) .$$

Similarly, construct

$$P_2 = (T[2:4]) (T[5:7]) (T[8:10]) \cdots (T[n'':n'' + 2]) ,$$

where n'' is the largest integer congruent to 2, modulo 3, that is less than or equal to n . For our example, we have

$$P_2 = (\text{ipp}) (\text{ity}) (\text{bop}) (\text{pit}) (\text{ybo}) (\text{o}\emptyset\emptyset) .$$

position in T	1	4	7	10	13	16	2	5	8	11	14	17
metacharacter in P	(bip)	(pit)	(ybo)	(ppi)	(tyb)	(ooø)	(ipp)	(ity)	(bop)	(pit)	(ybo)	(oøø)
character in P'	1	7	10	8	9	6	3	4	2	7	10	5
position in P'	1	2	3	4	5	6	7	8	9	10	11	12
$SA_{P'}$	1	9	7	8	12	6	10	2	4	5	11	3
positions in T of sorted sample suffixes of T	1	8	2	5	17	16	11	4	10	13	14	7

Figure 32.15 Computed values when sorting the sample suffixes of the linear-time suffix-array algorithm for the text $T = \text{bippityboppityboo}$.

If n is a multiple of 3, append the metacharacter $(\emptyset\emptyset\emptyset)$ to the end of P_1 . In this way, P_1 is guaranteed to end with a metacharacter containing $\emptyset$. (This property helps in part (a) of this problem.) The text P_2 may or may not end with a metacharacter containing $\emptyset$.

- B. Concatenate P_1 and P_2 to form a new text P . Figure 32.15 shows P for our example, along with the corresponding positions of T .
 - C. Sort and rank the unique metacharacters of P , with ranks starting from 1. In the example, P has 10 unique metacharacters: in sorted order, they are (bip), (bop), (ipp), (ity), (oøø), (ooø), (pit), (ppi), (tyb), (ybo). The metacharacters (pit) and (ybo) each appear twice.
 - D. As Figure 32.15 shows, construct a new “text” P' by renaming each metacharacter in P by its rank. If P contains k unique metacharacters, then each “character” in P' is an integer from 1 to k . The suffix arrays for P and P' are identical.
 - E. Compute the suffix array $SA_{P'}$ of P' . If the characters of P' (i.e., the ranks of metacharacters in P) are unique, then you can compute its suffix array directly, since the ordering of the individual characters gives the suffix array. Otherwise, recurse to compute the suffix array of P' , treating the ranks in P' as the input characters in the recursive call. Figure 32.15 shows the suffix array $SA_{P'}$ for our example. Since the number of metacharacters in P , and hence the length of P' , is approximately $2n/3$, this recursive subproblem is smaller than the current problem.
 - F. From $SA_{P'}$ and the positions in T corresponding to the sample positions, compute the list of positions of the sorted sample suffixes of the original text T . Figure 32.15 shows the list of positions in T of the sorted sample suffixes in our example.
2. The nonsample suffixes comprise about $1/3$ of the suffixes. Using the sorted sample suffixes, sort the nonsample suffixes by the following substeps.